

Install Linux WSL (bagi pengguna Windows) (diasarankan) [disini tutorial menggunakan linux Fedora]

Untuk pengguna MacOS juga disarankan menggunakan linux.

1. Install Ollama (linux)

curl -fsSL https://ollama.com/install.sh | sh

Apabila muncul peringatan seperti ini => ERROR: This version requires zstd for extraction. Please install zstd and try again:

Maka, install zstd terlebih dahulu dengan perintah:

sudo dnf install zstd

```
[gustipang@toolbox ~]$ curl -fsSL https://ollama.com/install.sh | sh
>>> Cleaning up old version at /usr/local/lib/ollama
>>> Installing ollama to /usr/local
>>> Downloading ollama-linux-amd64.tar.zst
##### 90.7%
```

2. Setelah Ollama terinstall, untuk menjalankannya ketikkan perintah “ollama” pada terminal. Setelah itu, kita dapat memilih model apa yang akan kita gunakan. Anggap kita menggunakan model secara offline, maka kita bisa memilih beberapa model offline, salah satunya adalah Mistral (ukuran yang ringan).

```
[gustipang@toolbox ~]$ ollama
Ollama 0.20.4

• Chat with a model
  Start an interactive chat with a model

  Launch Claude Code (not installed)
    Anthropic's coding tool with subagents

  Launch Codex (not installed)
    OpenAI's open-source coding agent

  Launch OpenClaw (install)
    Personal AI with 100+ skills

  More...
    Show additional integrations
```

Kita pilih menu paling atas, yaitu chat with a model. Setelah itu, kita bisa pilih model

(offline model) untuk kita install. Misalkan kita pilih model Gemma 4 (model terbaru yang lumayan ringan)

```
[gustipang@toolbox ~]$ ollama
Select model to run: Type to filter...

Recommended
  kimi-k2.5:cloud
    Multimodal reasoning with subagents
  qwen3.5:cloud
    Reasoning, coding, and agentic tool use with vision
  glm-5:cloud
    Reasoning and code generation
  minimax-m2.7:cloud
    Fast, efficient coding and real-world productivity
  ▸ gemma4
    Reasoning and code generation locally, ~16GB, (not downloaded)
  qwen3.5
    Reasoning, coding, and visual understanding locally, ~11GB, (not downloaded)

↑/↓ navigate • enter select • ← back
```

Klik enter, dan otomatis akan mendownload model tersebut

Note: namun perlu diketahui bahwa ukuran model juga berpengaruh kepada resource komputer yang digunakan termasuk penggunaan RAM. Untuk itu, pada kasus ini, kita menyarankan untuk menginstall model gemma 4 e2b dengan pertimbangan model yang memiliki ukuran kecil. Untuk itu, kita dapat exit proses yang ada dan menjalankan perintah sendiri:

```
ollama run gemma4:e2b
```

Nantinya, ollama akan mendownload model tersebut untuk dapat kita gunakan secara offline. (ukuran sekitar 7gb).

3. Setelah LLM terinstall, langkah berikutnya adalah menyiapkan environment Python. Untuk itu, pastikan kita install python pada wsl kita. Biasanya secara default, linux sudah terinstall python sejak pertama kali digunakan. Namun, versi yang dimiliki kebanyakan merupakan versi paling baru dari python (saat ini versi 3.14). Sehingga, terkadang ada beberapa tools yang belum terlalu update. Untuk itu, pada pelatihan ini kita akan menggunakan python dengan versi 3.12.
4. Langkah paling mudah untuk menginstall python versi 12 adalah dengan menggunakan pyenv, dengan menggunakan command:

```
curl -fsSL https://pyenv.run | bash
```

Setelah itu, seting environmen sehingga pyenv dapat digunakan oleh system.
Setelah pyenv dapat digunakan, langkah selanjutnya adalah menginstall python dengan versi 3.12 dengan perintah:

Pyenv install 3.12

Install CrewAI

Setelah python 3.12 sudah terinstall, langkah berikutnya adalah kita buat sebuah folder [project] dalam file Document, dengan cara

```
[gustipang@toolbox ~]$ cd Documents/  
[gustipang@toolbox Documents]$ mkdir project  
[gustipang@toolbox Documents]$
```

Nantinya folder bernama project akan terbuat didalam folder Documents.

Untuk membuat folder 'project' kita khusus hanya untuk akses python dengan versi 3.12.13, maka langkah selanjutnya adalah kita masuk ke dalam folder project melalui terminal lalu lakukan perintah:

pyenv global 3.12.13

Lalu, kita dapat menginstall crewAI dengan perintah:

curl -Lsf https://astral.sh/uv/install.sh | sh

Lalu, install dengan menggunakan perintah:

uv tool install crewai

Setelah itu, dengan menggunakan pyenv, kita buat virtual environment python dengan versi 3.12, menggunakan perintah:

Pyenv virtualenv 3.12.13 venv

Selanjutnya, folder venv akan terbentuk, folder ini berfungsi sebagai virtual environment python, sehingga tidak mempengaruhi environment python yang ada pada root system.

Fungsinya apabila kita ingin menginstall tools python, kita tidak akan melibatkan system root, sehingga root akan tetap aman sentosa.

Masih didalam folder project, Untuk menginstal crewAI, kita dapat terlebih dahulu mengaktifkan virtual env yang telah kita buat sebelumnya dengan perintah:

```
pyenv activate venv
```

Dengan begitu, folder project akan secara khusus menggunakan python versi 3.12.13 saja. Sedangkan folder dan alamat file lain masih tetap menggunakan python dengan versi global (3.14)

Untuk membuat project crewAI, kita dapat melakukan perintah ini di dalam folder project kita:

```
crewai create crew <your_project_name>
```

Setelah kita lakukan perintah diatas, maka project crewAI akan terbentuk. Kita dapat menggunakan IDE kita untuk edit project tersebut

Setelah itu, ada beberapa hal yang perlu kita install dengan menggunakan perintah uv, yaitu:

```
uv add fastapi uvicorn crewai celery redis
```

Dengan perintah di atas, maka uv akan menambahkan library dan juga tools seperti uvicorn, crewai, celery dan juga redis yang nantinya berguna untuk proses async dan pembuatan API.

*Note: jika menggunakan fedora ataupun ubuntu, maka kita juga perlu mengaktifkan redis dengan cara:

Linux (Ubuntu/Debian/Fedora)

bash

 Copy  Download

```
# Update package list and install Redis server
sudo apt update && sudo apt install redis-server -y # Ubuntu/Debian
# or
sudo dnf install redis # Fedora

# Start the Redis service and enable it to run on boot
sudo systemctl start redis-server
sudo systemctl enable redis-server

# Verify it's running and listening on the correct port
sudo systemctl status redis-server
redis-cli ping
```

macOS (using Homebrew)

bash

 Copy  Download

```
# Install Redis
brew install redis

# Start Redis as a background service
brew services start redis

# Verify it's running
redis-cli ping
```

Step 3: Alternative: Run Redis via Docker

If you prefer not to install Redis directly on your system, or want to keep your development environment clean, running Redis in a Docker container is a quick and reliable alternative.

bash

 Copy  Download

```
# Pull and run the official Redis container
docker run -d --name redis-stack -p 6379:6379 redis/redis-stack:latest
```


Agents

<https://docs.crewai.com/en/guides/agents/crafting-effective-agents>

```
writer:
  role: >
    Content Writer
  goal: >
    Write a compelling and well-structured blog post on {topic}
    based on the provided outline
  backstory: >
    You're a skilled writer with a talent for turning outlines into
    engaging, informative blog posts. Your writing is clear, conversational,
    and backed by solid reasoning. You adapt your tone to the subject matter
    while keeping things accessible to a broad audience.
```

The 80/20 Rule: Focus on Tasks Over Agents

- Spend most of your time writing clear task instructions
- Define detailed inputs and expected outputs
- Add examples and context to guide execution
- Dedicate the remaining time to agent role, goal, and backstory

Task

<https://docs.crewai.com/en/concepts/tasks>

```
planning_task:
  description: >
    Create a detailed outline for a blog post about {topic}.

    The outline should include:
    - A compelling title
    - An introduction hook
    - 3-5 main sections with key points to cover in each
    - A conclusion with a call to action

    Make the outline detailed enough that a writer can produce
    a full blog post from it without additional research.
  expected_output: >
    A structured blog post outline with a title, introduction notes,
    detailed section breakdowns, and conclusion notes.
  agent: planner
```


Celery

Celery adalah kerangka kerja antrean tugas (*task queue*) terdistribusi yang difokuskan pada pemrosesan tugas secara *real-time* maupun terjadwal. Sistem ini bekerja dengan mendistribusikan unit pekerjaan (tugas) ke berbagai *thread* atau mesin, di mana proses pekerja (*worker processes*) akan terus memantau antrean untuk mengeksekusi tugas baru yang masuk. Untuk saling berkomunikasi, klien dan *worker* menggunakan perantara pesan (*message broker*) seperti Redis atau RabbitMQ.

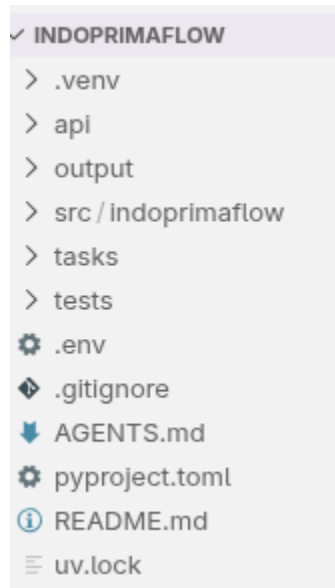
Dalam arsitektur REST API, khususnya saat membangun sistem agen dengan **CrewAI** (misalnya menggunakan FastAPI), Celery memiliki beberapa peran dan fungsi yang sangat krusial:

- **Mencegah Pemblokiran (Blocking) pada Server Web:** Menjalankan alur agen AI seperti `crew.kickoff()` secara langsung di dalam *route handler* HTTP merupakan sebuah *anti-pattern* dan kesalahan besar di lingkungan produksi. Pemanggilan API ke *Large Language Models* (LLM) oleh agen CrewAI bisa memakan waktu mulai dari 30 hingga 300 detik. Jika *server thread* menunggu proses agen selesai, ia tidak bisa melayani permintaan masuk lainnya dan akan dengan cepat menyebabkan koneksi *timeout*.
- **Menjalankan Proses di Latar Belakang (Asinkron):** Alih-alih membuat pengguna menunggu respons berlama-lama, REST API hanya akan mendelegasikan tugas ke Celery dengan menaruhnya di dalam antrean (misalnya memanggil `task.delay()`). API kemudian akan langsung memberikan respons ID Tugas (*job identifier*) kepada pengguna dalam waktu kurang dari 50 milidetik.
- **Pemantauan Status oleh Klien:** Sembari *worker* Celery secara terpisah menangani alur AI yang lambat tersebut, pihak klien (antarmuka depan/pengguna) dapat memantau status menggunakan ID tugas tadi melalui teknik *polling* (bertanya status secara berkala) atau *WebSockets* (menerima *push notification* jika pekerjaan telah selesai).
- **Penyimpanan Hasil Eksekusi:** Setelah eksekusi dari CrewAI selesai atau agen berhasil menjawab prompt yang diminta, *worker* Celery akan menyimpan hasil akhirnya di *result backend* seperti Redis atau Postgres. Endpoint status di REST API Anda nantinya akan mengambil hasil di basis data tersebut dan menampilkannya kepada klien.
- **Skalabilitas Horizontal:** Sistem ini memisahkan REST API (yang harus selalu cepat merespons) dan proses komputasi agen (yang berat). Anda dapat mengatur parameter jumlah komputasi secara paralel untuk tiap *worker* (berdasarkan beban I/O atau beban pemanggilan alat) atau memperbanyak *worker* Celery secara otomatis tanpa harus memberatkan *server API*.

Singkatnya, bisa diibaratkan REST API adalah staf "kasir" restoran cepat saji yang dengan lekas menerima pesanan (serta memberi nomor setruk tagihan), dan Celery adalah staf "dapur" di belakang layar yang perlahan dan cermat memasak pesanan LLM (CrewAI) tanpa mengganggu antrean kasir.

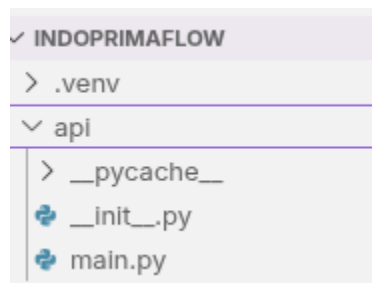
Langkah untuk menginstall celery:

1. Masuk ke root folder dengan terminal , lalu ketikan perintah:
`uv add "fastapi[all]" "celery[redis]" crewai crewai-tools`
2. Buat struktur file pada project seperti berikut:



Terdapat 2 folder tambahan yaitu "api" dan "task".

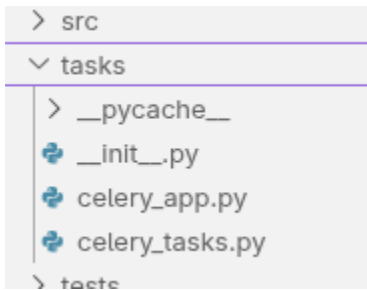
Folder "api" berisikan konfigurasi fastapi, untuk detilnya:



Folder __pycache__ akan secara otomatis muncul ketika program dijalankan. Yang perlu kita tambahkan secara manual adalah folder __init__.py (isinya kosong) dan main.py (berisi konfigurasi dari fastapi).

3. Sedangkan, untuk celery kita bisa membuat folder dengan nama "tasks", karena pada bagian celery inilah kita mengatur fungsi-fungsi apa saja yang nantinya dapat diakses oleh fastapi. Dengan adanya fungsi celery ini, request yang kita kirimkan

dapat berjalan secara paralel tanpa harus terjadi bottleneck. Struktur dari folder task sendiri adalah sebagai berikut:



Untuk `celery_app.py` sendiri berisikan pengaturan untuk celery, berupa:

```
import os
from celery import Celery

# Set the default Django settings module (if using Django)
# For a pure CrewAI project, we'll use a direct configuration
REDIS_URL = os.getenv("REDIS_URL", "redis://localhost:6379/0")

# Create Celery app
celery_app = Celery(
    "crewai_flow",
    broker=REDIS_URL,
    backend=REDIS_URL, # Use Redis as result backend as well
    include=["tasks"] # Import tasks module
)

# Optional: Configure Celery settings
celery_app.conf.update(
    task_serializer="json",
    accept_content=["json"],
    result_serializer="json",
    timezone="UTC",
    enable_utc=True,
    task_track_started=True,
    task_time_limit=30 * 60, # 30 minutes
    task_soft_time_limit=25 * 60, # 25 minutes
    worker_prefetch_multiplier=1, # Fair task distribution
)
```

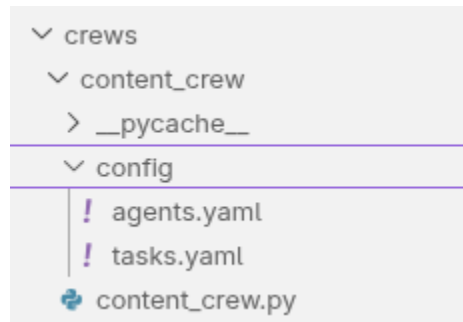
Sedangkan, untuk file `celery_tasks.py`, berisikan fungsi untuk memanggil agents ataupun crew (bahkan flow) pada crewai.

Contoh penggunaan celery untuk memanggil salah satu crew:

Misalkan, kita punya sebuah crew default dengan parameter sebagai berikut:

Nama crew: content_crew

Dengan struktur folder:



Agents.yaml

```
1  planner:
2    role: >
3      Content Planner
4    goal: >
5      Plan a detailed and engaging blog post outline on {topic}
6    backstory: >
7      You're an experienced content strategist who excels at creating
8      structured outlines for blog posts. You know how to organize ideas
9      into a logical flow that keeps readers engaged from start to finish.
10
11  writer:
12    role: >
13      Content Writer
14    goal: >
15      Write a compelling and well-structured blog post on {topic}
16      based on the provided outline
17    backstory: >
18      You're a skilled writer with a talent for turning outlines into
19      engaging, informative blog posts. Your writing is clear, conversational,
20      and backed by solid reasoning. You adapt your tone to the subject matter
21      while keeping things accessible to a broad audience.
22
23  editor:
24    role: >
25      Content Editor
26    goal: >
27      Review and polish the blog post on {topic} to ensure it is
28      publication-ready
29    backstory: >
30      You're a meticulous editor with years of experience refining written
31      content. You have an eye for clarity, flow, grammar, and consistency.
32      You improve prose without changing the author's voice and ensure every
33      piece you touch is polished and professional.
```

Tasks.yaml

```

planning_task:
  description: >
    Create a detailed outline for a blog post about {topic}.

    The outline should include:
    - A compelling title
    - An introduction hook
    - 3-5 main sections with key points to cover in each
    - A conclusion with a call to action

    Make the outline detailed enough that a writer can produce
    a full blog post from it without additional research.
  expected_output: >
    A structured blog post outline with a title, introduction notes,
    detailed section breakdowns, and conclusion notes.
  agent: planner

writing_task:
  description: >
    Using the outline provided, write a full blog post about {topic}.

    Requirements:
    - Follow the outline structure closely
    - Write in a clear, engaging, and conversational tone
    - Each section should be 2-3 paragraphs
    - Include a strong introduction and conclusion
    - Target around 800-1200 words
  expected_output: >
    A complete blog post in markdown format, ready for editing.
    The post should follow the outline and be well-written with
    clear transitions between sections.
  agent: writer

editing_task:
  description: >
    Review and edit the blog post about {topic}.

    Focus on:
    - Fixing any grammar or spelling errors
    - Improving sentence clarity and flow
    - Ensuring consistent tone throughout
    - Strengthening the introduction and conclusion
    - Removing any redundancy

    Do not rewrite the post – refine and polish it.
  expected_output: >
    The final, polished blog post in markdown format without `''`.
    Publication-ready with clean formatting and professional prose.
  agent: editor
  output_file: output/post.md

```

Lalu, kita buat sebuah class crew yang isinya adalah kolaborasi antar 3 agents dengan fungsi dan tugasnya masing-masing:

Content_crew.py

```

@CrewBase
class ContentCrew:
    """Content Crew"""

    agents: list[BaseAgent]
    tasks: list[Task]

    agents_config = "config/agents.yaml"
    tasks_config = "config/tasks.yaml"

    @agent
    def planner(self) -> Agent:
        return Agent(
            config=self.agents_config["planner"], # type: ignore[index]
        )

    @agent
    def writer(self) -> Agent:
        return Agent(
            config=self.agents_config["writer"], # type: ignore[index]
        )

    @agent
    def editor(self) -> Agent:
        return Agent(
            config=self.agents_config["editor"], # type: ignore[index]
        )

    @task
    def planning_task(self) -> Task:
        return Task(
            config=self.tasks_config["planning_task"], # type: ignore[index]
        )

    @task
    def writing_task(self) -> Task:
        return Task(
            config=self.tasks_config["writing_task"], # type: ignore[index]
        )

    @task
    def editing_task(self) -> Task:
        return Task(
            config=self.tasks_config["editing_task"], # type: ignore[index]
        )

```

```

@Crew
def crew(self) -> Crew:
    """Creates the Content Crew"""

    return Crew(
        agents=self.agents, # Automatically created by the @agent decorator
        tasks=self.tasks, # Automatically created by the @task decorator
        process=Process.sequential,
        verbose=True,
    )

```

Maka kita dapat memanggil def crew(self) pada celery_task.py dengan cara:

```

from tasks.celery_app import celery_app
from src.indoprimaflow.crews.crew_latihan.crew_latihan import CrewLatihan
from src.indoprimaflow.crews.content_crew.content_crew import ContentCrew
import logging
import traceback

logger = logging.getLogger(__name__)

@celery_app.task(bind=True, name="research")
def research(self, topic:str):
    self.update_state(state='RUNNING', meta={'current':f'start job for{topic}'})
    try:
        result = CrewLatihan().crew().kickoff_async(inputs = {"topic": topic})
        return str(result)
    except Exception as e:
        # self.update_state(state='FAILURE', meta={'error':str(e)})
        logger.error(f"Task failed with error: {e}\n{traceback.format_exc()}")
        raise

```

Setelah itu, kita dapat membuat rest api berdasarkan dari celery task yang telah kita buat, dengan cara:

```

from celery.result import AsyncResult
from tasks.celery_app import celery_app
from fastapi import FastAPI, UploadFile, File, BackgroundTasks, HTTPException
from pydantic import BaseModel
import tasks.celery_tasks as celeryTask

app = FastAPI()

class ResearchInput(BaseModel):
    topic: str

@app.post("/research")
async def research(researchInput: ResearchInput):
    task = celeryTask.research.delay(researchInput.topic)
    return {"task_id": task.id}

```

Menjalankan REST API

Sebelum menjalankan REST API, terlebih dahulu kita harus menjalankan servis masing-masing yang telah kita gunakan sebelumnya (Celery dan FastAPI), untuk menjalankan Celery, kita dapat menjalankan perintah pada terminal (root level):

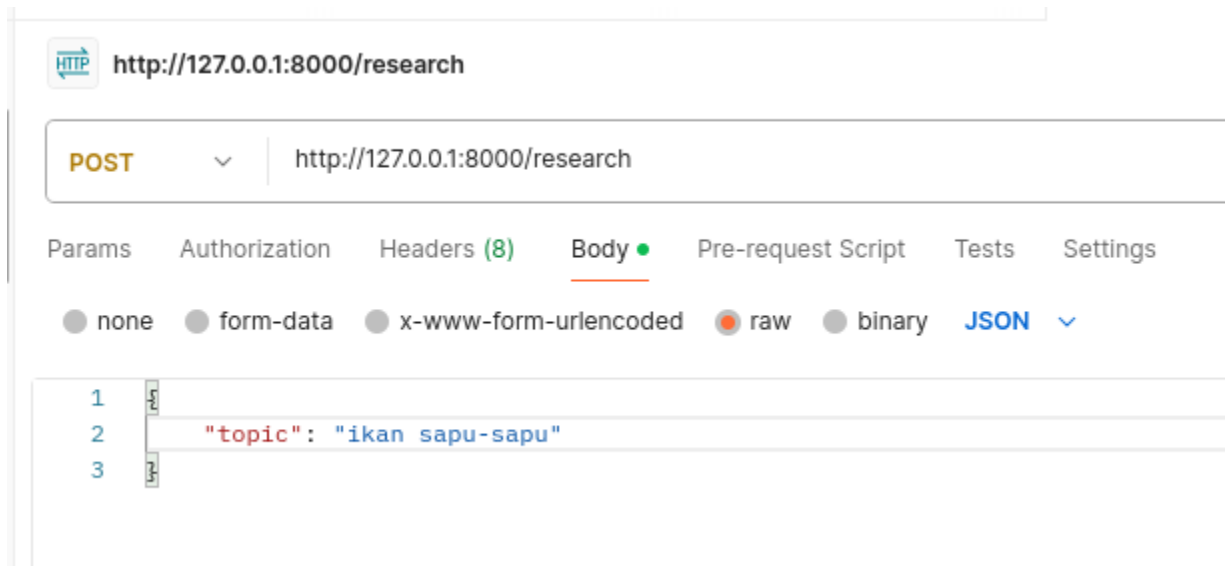
```
uv run celery -A [path to celery_task] worker --loglevel=info
```

Pada sistem operasi linux, kita diminta untuk menjalankan redis terlebih dahulu, baik dengan systemctl ataupun menggunakan servis seperti docker atau podman.

Dalam contoh kasus ini, saya menggunakan podman untuk menjalankan redis, dengan perintah:

```
podman run -d --name redis-server -p 6379:6379 -v redis-data:/data:Z redis redis-server --appendonly yes
```

Untuk menjalankan REST API, kita dapat menggunakan POSTMAN untuk melakukan pengecekan, misal kita akan memanggil API bernama “/research”, maka kita dapat lakukan:



Maka kita akan mendapatkan respon:



Respon yang kita terima merupakan sebuah ID proses yang sedang berjalan, untuk mengetahui hasilnya, kita dapat membuat rest api sebagai berikut:

```
@app.get("/status/{task_id}", response_model=TaskStatus)
async def get_status(task_id:str):
    task_result = AsyncResult(task_id, app=celery_app)

    response = {
        "task_id": task_id,
        "status": task_result.state,
        "result": None,
        "error": None
    }

    if task_result.state == 'SUCCESS':
        response["result"] = task_result.result
    elif task_result.state == 'FAILURE':
        response["error"] = str(task_result.info)

    return response
```

Setelah itu, dengan menggunakan postman, kita dapat lakukan proses cek:

 **http://127.0.0.1:8000/status/16643ae4-78ec-4c72-b862-258aa3b498b1**

GET

http://127.0.0.1:8000/status/16643ae4-78ec-4c72-b862-258aa3b498b1

Params

Authorization

Headers (6)

Body

Pre-request Script

Tests

Settings

Query Params

	Key
	Key

Body

Cookies

Headers (4)

Test Results

Pretty

Raw

Preview

Visualize

JSON

1

2

3

4

5

6

"task_id": "16643ae4-78ec-4c72-b862-258aa3b498b1",

"status": "RUNNING",

"result": null,

"error": null

Lalu, apabila agent selesai, maka kita bisa mendapatkan hasil:

```
"task_id": "16643ae4-78ec-4c72-b862-258aa3b498b1",
"status": "SUCCESS",
"result": "The Hidden Gem of the Waters: A Complete Guide to Ikan Sapu-Sapu\n\nHave you ever stopped to consider the incredible depth and diversity of the ocean's bounty? Today, we delve into the world of marine life, exploring the nuances of a fascinating species.\n\n***\n\n## The Deep Dive: An Exploration of Ikan Sapu\n\nThe world beneath the waves is a tapestry of complex ecosystems, each woven with unique biological stories. Understanding the life and habitat of creatures like *Ikan Sapu—a fish known for its unique ecological role—is vital to appreciating the health of our planet's oceans. This exploration seeks to illuminate the interconnectedness of marine life, from the food web to the vital role of conservation.\n\n\n\n\n\n## Unraveling the Ecosystem\n\nThe marine environment is a dynamic system where every organism plays a crucial role. The health of a single species directly impacts the stability of the entire ecosystem.\n\n\n\n\n\n## The Interconnected Web\n\nEcosystems thrive on intricate food webs where producers, consumers, and decomposers interact in a delicate balance. Disrupting this balance, whether through pollution or overfishing, can trigger a cascade effect that harms the entire marine environment.\n\n\n\n\n\n## The Role of Biodiversity\n\nThe sheer variety of life, or biodiversity, is the foundation of a resilient ocean. Each species, no matter how small, contributes a unique function—providing food, cycling nutrients, and maintaining habitat structure. Protecting this biodiversity is not just about saving individual species; it is about safeguarding the entire ecological health of the sea.\n\n\n\n\n\n## Conservation in Action\n\nEffective conservation strategies must be holistic, addressing threats at the local, regional, and global levels. This involves managing fishing practices sustainably, controlling pollution, and protecting critical habitats like coral reefs and mangrove forests. By supporting responsible management, we ensure that the rich biodiversity of the ocean endures for future generations.\n\n\n\n\n\n\n\n## A Call for Stewardship\n\nThe ocean is not just a source of resources; it is a living entity that sustains human
```